

High Level Design - Architecture

- **Project Description:**

Compare a student's current skill profile with a job's expected skill profile.

Identify the skill gaps.

Map those gaps to courses.

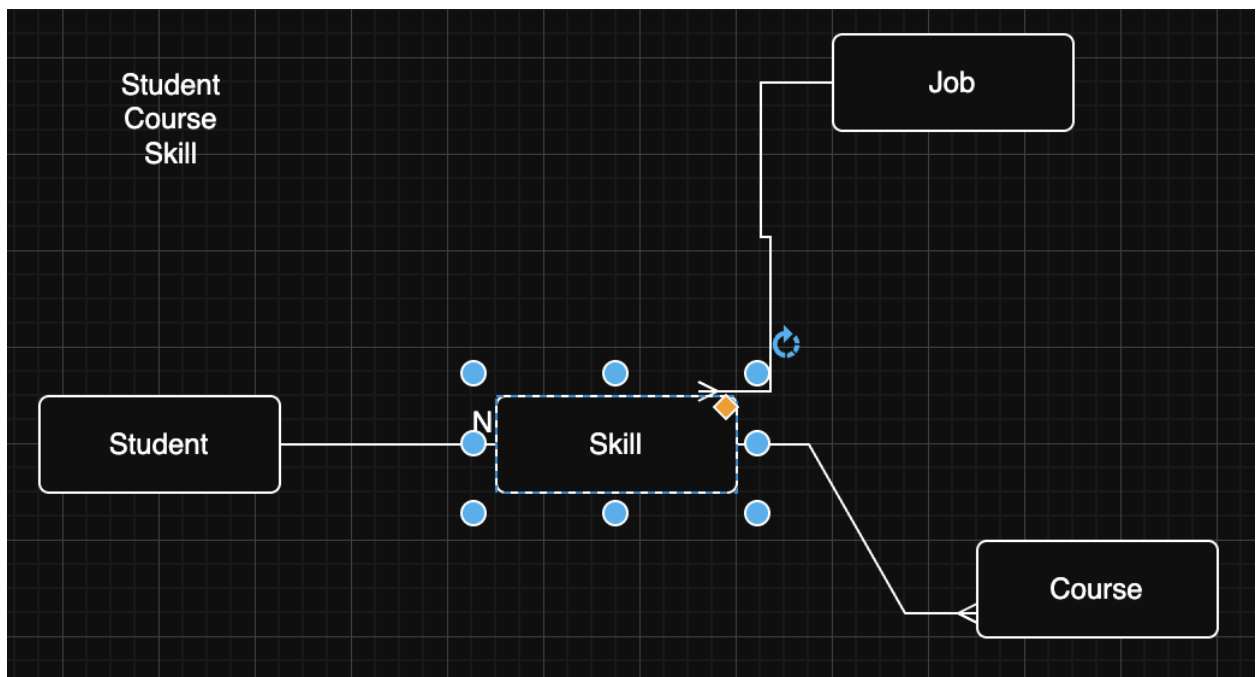
Rank the courses based on **time and cost efficiency**.

Present the output in a **React page** with a **spider/radar chart visualization**.

Use **LangChain + RAG** for reasoning and recommendation.

Optionally use **Model Context Protocol (MCP)** to share tabular data with the LLM in a structured way

- **Entity Relationship:**



- **Student**

- `student_id` (PK)
- `name`
- `email`
- `current_skill_level` (optional if kept in mapping table)
- **Job**
 - `job_id` (PK)
 - `title`
 - `description`
- **Skill**
 - `skill_id` (PK)
 - `name`
 - `category`
- **StudentSkill** (many-to-many mapping between Student & Skill)
 - `student_id` (FK → `Student.student_id`)
 - `skill_id` (FK → `Skill.skill_id`)
 - `level` (e.g., 1–10 scale)
- **JobSkill** (many-to-many mapping between Job & Skill)
 - `job_id` (FK → `Job.job_id`)
 - `skill_id` (FK → `Skill.skill_id`)
 - `required_level`

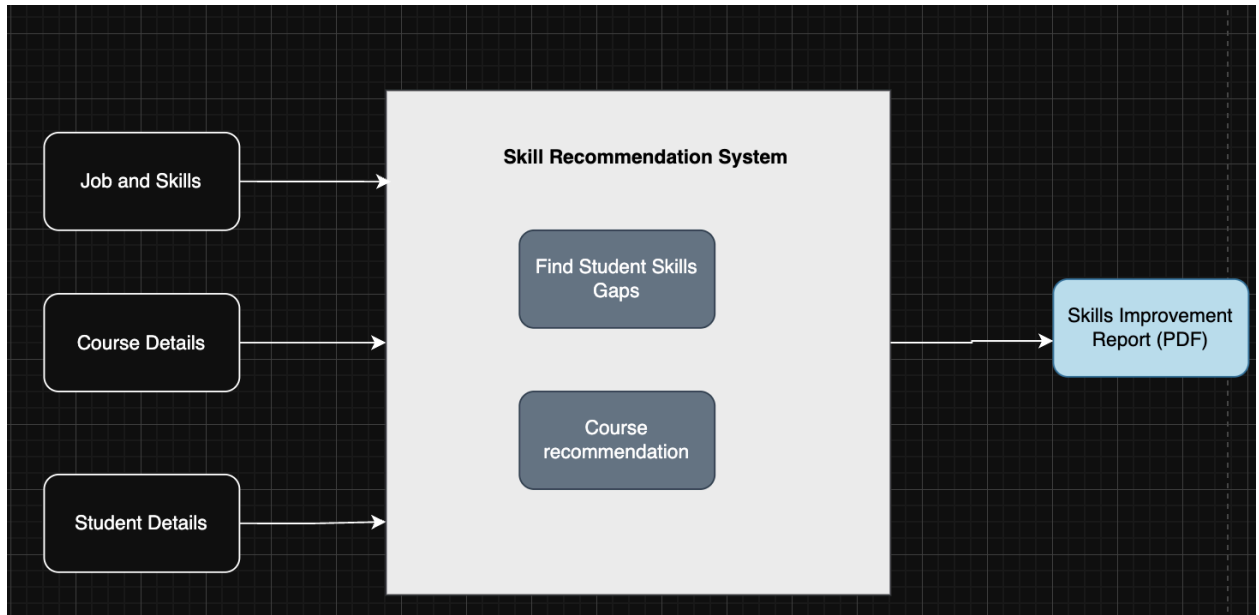
- **Course**

- `course_id` (PK)
- `name`
- `provider`
- `duration_hours`
- `cost`
- `skill_id` (FK → `Skill.skill_id`)
- `improvement_level` (how much it can raise the skill score)

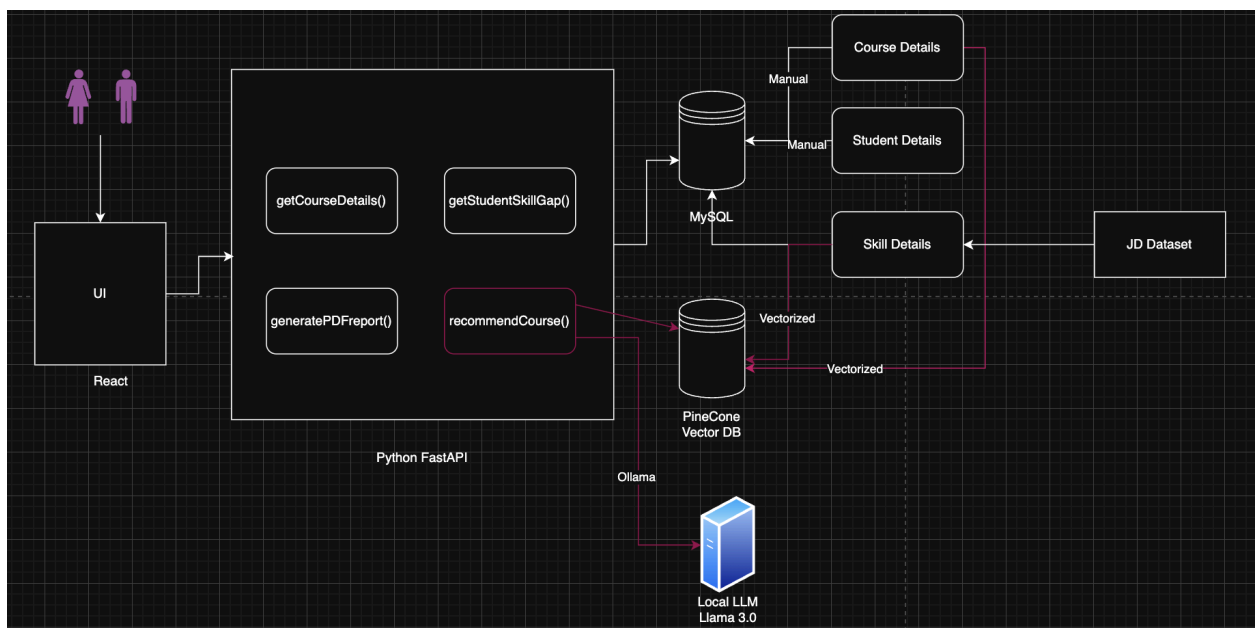
- **Recommendation** (generated)

- `recommendation_id` (PK)
- `student_id` (FK)
- `job_id` (FK)
- `course_id` (FK)
- `priority_score` (based on cost & time effectiveness)

- Logical Architecture:



- Technical Architecture:



- Table Structure:

Student

CREATE TABLE Student (

```
        student_id INT PRIMARY KEY,  
        name VARCHAR(100) NOT NULL  
    );
```

Skill

```
CREATE TABLE Skill (  
    skill_id INT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    domain VARCHAR(100)  
);
```

Job

```
CREATE TABLE Job (  
    job_id INT PRIMARY KEY,  
    title VARCHAR(100) NOT NULL,  
    description TEXT  
);
```

Student_Skill

```
CREATE TABLE Student_Skill (  
    student_id INT,  
    skill_id INT,  
    curr_lvl INT CHECK(curr_lvl BETWEEN 0 AND 5),  
    PRIMARY KEY(student_id, skill_id),  
    FOREIGN KEY(student_id) REFERENCES Student(student_id),  
    FOREIGN KEY(skill_id) REFERENCES Skill(skill_id)  
);
```

Job_Skill

```
CREATE TABLE Job_Skill (  
    job_id INT,  
    skill_id INT,  
    target_lvl INT CHECK(target_lvl BETWEEN 0 AND 5),  
    PRIMARY KEY(job_id, skill_id),  
    FOREIGN KEY(job_id) REFERENCES Job(job_id),
```

```
        FOREIGN KEY(skill_id) REFERENCES Skill(skill_id)
    );
```

Course

```
CREATE TABLE Course (
    course_id INT PRIMARY KEY,
    title VARCHAR(200) NOT NULL,
    skill_id INT NOT NULL,
    level_gain INT CHECK(level_gain >= 1),
    duration INT,    -- hours
    cost DECIMAL(10,2),
    provider VARCHAR(100),
    FOREIGN KEY(skill_id) REFERENCES Skill(skill_id)
);
```

- API Specification:

- 1. Students

- GET /students/{student_id}**

- **Description:** Fetch student details along with current skill levels
 - **Response:**

```
{  
  
  "student_id": 1,  
  
  "name": "Shriram",  
  
  "email": "shri@example.com",  
  
  "skills": [  
  
    {"skill_id": 101, "name": "Python", "level": 6},  
  
    {"skill_id": 102, "name": "SQL", "level": 5}  
  
  ]  
  
}
```

- POST /students**

- **Description:** Create a new student record
 - **Request:**

```
{
```

```
{
  "name": "Shriram",
  "email": "shri@example.com",
  "skills": [
    {
      "skill_id": 101,
      "level": 6
    },
    {
      "skill_id": 102,
      "level": 5
    }
  ]
}
```

- **Response:** 201 Created

```
{
  "student_id": 1,
  "message": "Student created successfully"
}
```

PUT /students/{student_id}

- **Description:** Update student profile or skill levels
- **Request:**

```
{
  "name": "Shriram",
  "email": "shri@example.com",
  "skills": [
    {
      "skill_id": 101,
      "level": 7
    },
    {
      "skill_id": 103,
      "level": 4
    }
  ]
}
```



```
}
```

- **Response:**

```
{"message": "Student updated successfully"}
```

2. Jobs

GET /jobs/{job_id}

- **Description:** Fetch job details and required skills

- **Response:**

```
{  
  
  "job_id": 501,  
  
  "title": "Data Analyst",  
  
  "description": "Analyze datasets to provide insights",  
  
  "skills_required": [  
  
    {"skill_id": 101, "name": "Python", "required_level": 7},  
  
    {"skill_id": 102, "name": "SQL", "required_level": 6}  
  
  ]  
  
}
```

POST /jobs

- **Description:** Add a new job role with skill requirements
- **Request:**

```
{  
  
  "title": "Data Analyst",  
  
  "description": "Analyze datasets to provide insights",  
  
  "skills_required": [  
  
    {"skill_id": 101, "required_level": 7},  
  
    {"skill_id": 102, "required_level": 6}  
  
  ]  
  
}
```

- **Response:**

```
{"job_id": 501, "message": "Job created successfully"}
```

3. Skills

GET /skills

- **Description:** Get full catalog of skills
- **Response:**

```
[  
  
  {"skill_id": 101, "name": "Python", "category": "Programming"},  
  
  {"skill_id": 102, "name": "SQL", "category": "Database"}  
  
]
```

POST /skills

- **Description:** Add a new skill
- **Request:**

```
{"name": "Tableau", "category": "Visualization"}
```

- **Response:**

```
{"skill_id": 103, "message": "Skill created successfully"}
```

4. Courses

GET /courses

- **Description:** Fetch available courses, filter by skill, cost, duration

- **Query Params:**

- `skill_id` (optional)
- `max_cost` (optional)
- `max_duration` (optional)

- **Response:**

```
[  
  
  {  
  
    "course_id": 201,  
  
    "name": "Intro to Python",  
  
    "provider": "Coursera",  
  
    "duration_hours": 20,  
  
    "cost": 50,  
  
    "skill_id": 101,  
  
    "improvement_level": 2  
  
  }  
  
]
```

POST /courses

- **Description:** Add new course

- **Request:**

```
{  
  
  "name": "Advanced SQL",  
  
  "provider": "Udemy",  
  
  "duration_hours": 15,  
  
  "cost": 40,  
  
  "skill_id": 102,  
  
  "improvement_level": 2  
  
}
```

- **Response:**

```
{"course_id": 202, "message": "Course created successfully"}
```

5. Recommendations

POST /recommendations

- **Description:** Suggest courses for a student applying for a job
- **Request:**

```
{"student_id": 1, "job_id": 501}
```

- **Response:**

```
{  
  
  "student": "Shriram",  
  
  "job": "Data Analyst",  
  
  "skill_gap": [  
  
    {"skill": "Python", "current_level": 6, "required_level": 7,  
    "gap": 1},  
  
    {"skill": "SQL", "current_level": 5, "required_level": 6,  
    "gap": 1}  
  
  ],  
  
  "recommended_courses": [  
  
    {  
  
      "course_id": 201,  
  
      "name": "Intro to Python",  
  
      "provider": "Coursera",  
  
      "duration_hours": 20,  
  
      "cost": 50,  
  
      "priority_score": 0.85  
  
    },  
  
  ],  
  
}
```

```
{  
  "course_id": 202,  
  "name": "Advanced SQL",  
  "provider": "Udemy",  
  "duration_hours": 15,  
  "cost": 40,  
  "priority_score": 0.90  
}  
]  
}
```

6. Reports

GET /reports/{student_id}/{job_id}

- **Description:** Generate a PDF skill-gap & course recommendation report (via WeasyPrint backend)
- **Response:** PDF file download

- UI Wireframe:

The wireframe is set against a dark gray grid background. At the top left, the text "Student ID" is positioned above a white rectangular input field. Below this, the text "Job Description" is positioned above a dropdown menu. The dropdown menu is white with the text "Data Analyst" and a blue downward-pointing triangle icon. A vertical dashed line runs down the right side of the form. At the bottom, there is a table with five columns. The first column is labeled "Student ID", the second "Student Name", the third "Student target Skill", the fourth "Student Current Skill", and the fifth "Knowledge Gap". To the right of the table, there is a blue button with the text "Suggest Course".

Student ID	Student Name	Student target Skill	Student Current Skill	Knowledge Gap
------------	--------------	----------------------	-----------------------	---------------

Suggest Course